

Overview — Paperclip은 무엇인가

1. 한 줄 요약

Paperclip은 AI 에이전트들로 구성된 회사를 운영하기 위한 오픈소스 **control plane**이다. “회사” 단위로 에이전트들이 조직되고, 작업이 단일 골(goal)을 향해 위계적으로 분해되며, 사람(보드 운영자)은 승인·예산·일시정지 권한으로 거버넌스를 행사한다.

저장소의 슬로건은 핵심을 압축한다 — “If OpenClaw is an *employee*, Paperclip is the *company*.” 즉 Paperclip은 **에이전트 자체**가 아니라, **에이전트를 고용·조직·예산 통제·감사하는 회사 운영체제**다. 코어 서버는 LLM 호출/툴 사용 같은 *에이전트 런타임 로직*을 내장하지 않으며, 그 역할은 어댑터(외부 CLI·웹훅·원격 게이트웨이)가 맡는다.

2. 풀려는 문제

저장소 README([README.md:131-140](#))가 “Problems Paperclip solves” 표로 든 출발점은 여섯 가지다.

Architecture — 모노레포 구조

1. 모노레포 한눈에

Paperclip은 **pnpm workspace** 모노레포다. 루트의 `pnpm-workspace.yaml` 이 워크스페이스를 선언하고, 각 패키지는 `workspace:*` 프로토콜로 서로를 참조한다. 그림 1-1은 패키지 지도와 의존 방향을 한 페이지로 보여 준다. 의존 화살표는 “위쪽 패키지가 아래쪽 패키지를 import 한다”를 의미한다.

그림 1-1. pnpm 모노레포 패키지 지도와 의존 방향

그림 1-1의 구조는 세 계층으로 나뉜다. (1) 최상위 **애플리케이션** — `server/`, `ui/`, `cli/`. (2) **공유 코어** — `packages/db`, `packages/shared`, `packages/mcp-server`. (3) **확장점** — `packages/adapters/*`, `packages/adapters/adapter-utils`, `packages/plugins`. 외부에는 사용자가 설치하는 **어댑터 플러그인**이 점선으로 그려져

있는데, 이는 코어를 수정하지 않고 새로운 에이전트 런타임을 끼워 넣는 통로다. 의존이 위에서 아래로만 흐른다는 점도

그림에서 확인할 수 있다 — `packages/shared`가 `server/`를 import 하지 않고, 하위 패키지는 상위 패키지를 참조하지

Data Model — 80개 이상 테이블 한눈에 보기

1. 데이터 모델 한눈 — 핵심 ER

`packages/db/src/schema/`에는 86개의 `pgTable()` 정의가 78개 스키마 파일에 분산되어 있다 (`packages/db/src/schema/index.ts:1-78`). 모두를 한 그림에 담는 것은 무의미하므로, 보드 운영 흐름을 따라가는 데 필수적인 12~13개 테이블만 추려 그림 4로 묶었다.

그림 4. 핵심 ER 다이어그램 — issues를 중심으로 한 12개 이상 테이블

가운데 빨간 박스가 **issues** — 단일 작업 단위(unit of work)다. 위쪽 회색 박스 4개(`agents` · `goals` · `projects` · `approvals`)는 모두 `companies`에서 내려온다. 아래쪽 청색 박스 그룹(`cost_events` · `budget_policies` · `budget_incidents` · `activity_log`)은 비용·거버넌스 레이어다. 이 그림은 모든 비즈니스 데이터가 `company_id`로 **스코프**되며, 이슈 한 개가 어떻게 cost·activity·heartbeat와 연결되는지를 보여 준다. 핵심 관찰점은 세 가지다. 첫째,^{3 / 41}

`issues` 박스 안의 `assignee_agent_id XOR assignee_user_id` 한 줄은 한 이슈를 오직 한 명의 에이전트 또는 사람이

1) 스키마 코드를 편집

\$EDITOR packages/db/src/schema/issues.ts

2) SQL 마이그레이션 자동 생성

pnpm db:generate

3) 마이그레이션 적용

```
pnpm db:migrate ```
```

생성된 마이그레이션은 `packages/db/src/migrations/` 아래에 누적된다(`packages/db/drizzle.config.ts:5`).
그림 5 는 Drizzle 팀이 공식적으로 제공하는 Drizzle Studio 스크린샷으로, 스키마 시각화와 데이터 편집을 같은 도구에서 할 수 있음을 보여 준다(자세한 내용은 [docs/research/06-drizzle-pglite.md](https://docs.drizzle.team/06-drizzle-pglite.md)).

그림 5. Drizzle Studio (출처: orm.drizzle.team/drizzle-studio/overview)

8. 임베디드 PostgreSQL — 무설정 dev

`DATABASE_URL` 이 비어 있으면 Paperclip 은 자동으로 `~/.paperclip/instances/default/db/` 에 **embedded PostgreSQL** 인스턴스를 띄운다. 이 패키지는 `package.json` 에서 `embedded-postgres@18.1.0-beta.16` 으로 AI Odyssey ⁶ / 41 (pin)되어 있고, 한 줄짜리 패치(`patches/embedded-postgres@*.patch`) 가 적용된다. PostgreSQL 17 (full native) 을

Docker Postgres

DATABASE_URL=postgres://paperclip:paperclip@localhost:5432/paperclip

Supabase direct / session (5432) — DDL·마이그레이션에 적합

DATABASE_URL=postgres://postgres.[ref]:[pw]@aws-0-[region].pooler.supabase.com:5432/postgres

Supabase pooled runtime (6543) + 마이그레이션은 5432 직결

```
DATABASE_URL=postgres://postgres.[ref]:[pw]@aws-0-[region].pooler.supabase.com:6543/postgres
DATABASE_MIGRATION_URL=postgres://postgres.[ref]:[pw]@aws-0-
[region].pooler.supabase.com:5432/postgres ````
```

9. 요약

Paperclip의 데이터 모델은 회사 스코프, 단일 assignee, atomic checkout, 비용 귀속, 감사 로그를 중심으로 구성된다. [03-runtime-execution.md](#)는 이 모델 위에서 heartbeat scheduler · atomic checkout · stranded-work 회복 · silent-run watchdog이 어떻게 동작하는지 분석한다.

Runtime Execution — Heartbeat

• Checkout • Watchdog

1. 한 회차의 heartbeat가 만드는 일들

Paperclip의 실행 의미론은 한 가지 단순한 그림으로 압축된다 — **scheduler/wake claim** → **server-side atomic checkout** → **adapter.execute** → **agent runs** → **cost report** → **live event push** 의 직선이다. 그림 3-1이 6개 라이프라인(Scheduler · Adapter · Agent runtime · Paperclip API · PostgreSQL · WebSocket→UI) 위에 그 흐름을 시간 축으로 펼쳐 보여 준다.

그림 3-1. heartbeat 한 회차의 시퀀스 — 6 라이프라인을 가로지르는 $t_0 \sim t_6$ 흐름

t_0 에 wake 큐가 만기 도래한 에이전트의 run을 클레임한다. t_1 에 Paperclip server-side에서 atomic checkout이 이슈를 잡는다(`server/src/services/issues.ts:4659-4809`). t_2 에 서버가 `adapter.execute(ctx)` 를 호출해 외부

Adapters & Skills — built-in 어댑터 + Paperclip Skill

1. “호출 가능한 것은 모두 채용된다”

Paperclip README의 슬로건 — “*If it can receive a heartbeat, it’s hired.*” — 은 어댑터 시스템의 핵심 약속이다. 어댑터는 Paperclip control plane과 외부 에이전트 런타임 사이의 좁은 통로다.

그림 7 이 런타임 전용 built-in 어댑터 11개와 범용 어댑터 2개(`process` · `http`)를 능력 축으로 정렬한다 (`server/src/adapters/registry.ts:511-524`). 각 축은 `ServerAdapterModule` (`packages/adapters/adapter-utils/src/types.ts:349-431`)의 실제 필드 — 핵심 실행·모델·옵션 필드(`execute` , `testEnvironment` , `listSkills` / `syncSkills` , `sessionCodec` , `supportsLocalAgentJwt` , `getQuotaWindows` , `detectModel` , `getConfigSchema` 등은 `:349-396`)와 capability flags(`supportsInstructionsBundle` , `instructionsPathKey` , `requiresMaterializedRuntimeSkills` , `getRuntimeCommandSpec` 등은 `:407-431`) — 에 대응한다.

1) 어댑터 플러그인을 설치

```
pnpm paperclipai plugin install
```

2) 플러그인이 ~/.paperclip/adapter- plugins.json 에 등록

3) 보드 UI > Adapter manager 에서 활성화

4) 새 에이전트 만들 때 adapter_type=

5.1 Plugin-managed skills — 플러그인이 회사 스킬을 선언·재조정

정적 `skills/` 폴더 외에, 플러그인이 회사 스킬을 런타임에 선언·생성·재조정할 수 있는 통로가 있다.

`server/src/services/plugin-managed-skills.ts:1-359` 의 `pluginManagedSkillService` 가 그 본체로, `skills.managed` capability를 가진 plugin manifest의 `skills` 배열(schema는 `packages/shared/src/validators/plugin.ts:246-269` + `:624-628`, SDK re-export는 `packages/plugins/sdk/src/index.ts:260-273`)을 받아 `plugin_managed_resources` 테이블에 binding을 만들고, `get / reconcile / reset` 흐름(`packages/plugins/sdk/src/types.ts:852-862`)으로 회사 스킬과 동기화한다. 실제 사례는 `packages/plugins/plugin-llm-wiki/` 가 6개의 managed skill을 선언한다 — wiki 검색·편집·인덱싱·소비 등. 즉 회사가 어떤 스킬을 가지는가는 정적 `skills/` 만이 아니라 설치된 플러그인이 무엇을 declare하는가에 따라서도 결정된다.

Server & API — Express · Routes · Services · WebSocket

1. 한 호스트, 두 종류의 클라이언트

Paperclip API의 첫 번째 결정은 단순하다 — 하나의 REST 표면, 두 가지 인증이다. 같은 `/api/...` 엔드포인트가 (1) 보드 운영자(웹 UI · CLI)와 (2) 에이전트(API key)를 동시에 받는다. 어떤 키로 들어왔는가가 권한을 결정한다. 이는 SPEC § 8의 “*Single unified REST API*” 결정이다.

그림 3이 `server/src`의 내부 토폴로지를 한 페이지로 그려 준다.

그림 3. `server/src`의 내부 토폴로지 — `auth` · `middleware` · `realtime` · `routes` · `services` · `adapters` · `DB`

AI Odyssey inbound(`app.ts` · `auth/` · `middleware/` · `realtime/`)에서 들어와 → `routes/`로 분기 → `services/`가 도메인¹⁶ / 41

르지는 단단 → `adapters/`가 외부 런타임을 부르고 → `DB` · `storage` · `secrets`가 연속화한다. 모든 mutating 작업은

보드 UI (브라우저 same-origin)

`new WebSocket( ${proto}://${host}/api/companies/:companyId/events/ws )` → upgrade 시 별도 헤더 없음.
authorizeUpgrade가 Better Auth 세션 또는 local_trusted board context 를 검증.

에이전트 (헤드리스)

GET wss:///api/companies/:companyId/events/ws Authorization: Bearer → authorizeUpgrade가 agent_api_keys 조회로 검증.

이 통로 덕분에 지식 베이스 · 매출 회계 · 새 어댑터 같은 V1 범위 밖 기능을 코어 수정 없이 도입할 수 있다.

8. 요약

Paperclip의 서버는 “한 표면, 두 사용자, 5개 서비스 도메인”이다. Express 라우트는 얇고, services/가 두껍고, 라이브 이벤트는 회사 단위 WebSocket 한 줄로 흐르며, 플러그인 런타임이 코어 수정 없는 확장을 허용한다.

[06-ui-and-board.md](#)는 보드 UI — React + Vite + shadcn/ui — 가 이 표면을 어떻게 그리는지 분석한다.

UI & Board — React · Vite · shadcn/ui

1. UI의 위치

`ui/` 는 보드 운영자가 일상적으로 머무는 single page application 이다. 기술 스택은 다음과 같다
(`ui/package.json:30-78`) — **Vite 6 + React 19 + TypeScript + Tailwind CSS 4 + shadcn/ui + Radix UI + TanStack Query 5 + react-router-dom 7**. 같은 프로젝트에 Storybook 이 별도 디렉터리로 동거(`ui/storybook/`) 한다. 프론트엔드 기술 토대의 자세한 설명은 [docs/research/08-react-vite-shadcn.md](https://docs.research/08-react-vite-shadcn.md).

shadcn/ui 의 공식 **dashboard-01** 블록 은 이 톤의 표준 모범이다 — Paperclip 보드 UI 는 그 위에 회사·이슈·코스트 도메인을 입혔다. 그림 6-1 은 *외부 인용 다이어그램* 으로, Paperclip 자체 스크린샷이 아니라 디자인 톤의 *원형* 을 보여 준다 — 좌측 사이드바 + 상단 헤더 + 그리드 카드 위주의 대시보드 레이아웃이 Paperclip `/dashboard` 와 1:1 로 이어진다.

그림 6-1. shadcn/ui 공식 dashboard-01 블록 (출처: ui.shadcn.com)

Governance & Cost — Board · Approvals · Budget · Activity Log

1. 사람이 떠나지 않는 자리

자율 에이전트 회사의 핵심 이슈는 *얼마나 자율인가*가 아니라 *어디서 인간이 결정해야 하는가*다. Paperclip은 그 자리를 **Board**라는 이름으로 명시한다. SPEC § 1은 다음을 포함한다.

- **승인 게이트** — 신규 에이전트 고용, CEO의 초기 전략 분해
- **항상 가용한 권한** — 회사 예산 설정, 임의의 에이전트/이슈/프로젝트 일시정지·재개, 풀 PM 권한, 모든 에이전트 결정의 override, 임의 단계의 예산 수정

V1의 보드는 *단일* 인간 운영자다(SPEC § 10의 *Single human Board*).

Usage — 설치 · 회사 만들기 · 첫 운영

1. 사전 준비

요구	버전	비고
Node.js	≥ 20 (LTS)	<code>package.json engines</code> 강제
pnpm	≥ 9	<code>packageManager: pnpm@9.15.4</code>
OS	macOS · Linux · Windows	embedded-postgres 18 native 바이너리가 깔린다
(옵션) Docker	24+	local Docker Postgres 모드용
(옵션) Tailscale	—	private 네트워크 모드용

→ **http://localhost:3100**

onboard 가 묻는 첫 질문은 reachability다(자세한 내용 doc/DEPLOYMENT-MODES.md § 4). 코드 3은 4가지 reachability 옵션이며, 1번이 *나만 쓴다*, 2\~3번이 *팀이 쓴다*, 4번이 *외부에 공개한다*는 운영 톤에 대응한다.

코드 3. onboard reachability 4가지 옵션

4. 보드 UI 첫 화면 — 회사를 만들자

UI가 처음 띄우는 흐름은 두 갈래다.

(A) 새 회사 만들기 - Company name — 첫 화면 필수 입력 - Mission / goal — 선택, 비워 둘 수 있음 (e.g., “*Build the #1 AI note-taking app to \$1M MRR.*”)

첫 화면은 이 두 입력으로 끝난다(ui/src/components/OnboardingWizard.jsx:681-700) description :

1) 회사 상태

```
curl -s http://localhost:3100/api/health curl -s http://localhost:3100/api/companies
```

2) dev 러너 인스턴스 관리

`pnpm dev:list` `pnpm dev:stop`

3) 마이그레이션 수동 적용

pnpm db:migrate

4) 전체 검증 (PR 전 단계)

```
pnpm -r typecheck pnpm test:run pnpm build
```

5) E2E (UI 흐름이 바뀌었을 때만)

```
pnpm test:e2e pnpm test:release-smoke ```
```

AGENTS.md § 7의 권고는 “PR 핸드오프 전에는 전체 체크, 그 외에는 가장 좁은 검증”이다.

8. Docker 한 줄로 띄우기 (옵션)

코드 5는 두 가지 Docker 사용 패턴이다 — `docker compose` 로 PostgreSQL 17 + Paperclip server 풀스택을 한 명령으로 띄우거나, `docker build` + `docker run` 으로 호스트 DB(또는 임베디드 Postgres) 위에 Paperclip 컨테이너만 띄우거나. 후자가 완전 *portable*이라 운영 이전이 가장 단순.

코드 5. Docker 사용 패턴 — full-stack compose vs Paperclip 단독 컨테이너

```
```bash
```

# 풀스택 (PostgreSQL 17 + Paperclip server). compose 가 둘 다 띄운다

```
BETTER_AUTH_SECRET=$(openssl rand -hex 32) \ docker compose -f docker/docker-compose.yml up -build
```

## 또는 Paperclip 컨테이너만 (임베디드 Postgres 모드)

```
docker build -t paperclip:dev . docker run -p 3100:3100 \ -e BETTER_AUTH_SECRET=$(openssl rand -hex 32) \
-v paperclip-data:/paperclip paperclip:dev ``
```

위 compose 는 `db` (postgres:17-alpine) 와 `server` (Paperclip Dockerfile) 두 서비스를 함께 띄우며 `BETTER_AUTH_SECRET` 이 강제된다( `docker/docker-compose.yml:18-31` , `doc/DOCKER.md:84-89` ). 단독 컨테이너 경로도 마찬가지로 `BETTER_AUTH_SECRET` 이 필요한데, 루트 `Dockerfile` 의 기본값이 `PAPERCLIP_DEPLOYMENT_MODE=authenticated` ( `Dockerfile:76` )이기 때문이다. 단독 컨테이너는 임베디드 Postgres 모드를 그대로 쓰므로 `/paperclip` 볼륨만 보존하면 된다( `doc/DOCKER.md` ).

## 9. 호스팅 시나리오 — Supabase + Vercel

규모가 커지면 호스팅으로 옮긴다( `doc/DATABASE.md` § 3). **코드 6** 은 Supabase + Vercel 같은 매니지드 환경으로 옮길 때 필요한 핵심 환경 변수다 — DB/마이그레이션·secrets·auth 세 묶음으로 나뉜다.

# DB / 마이그레이션

DATABASE\_URL=postgres://postgres.[ref]:[pw]@aws-0-[region].pooler.supabase.com:5432/postgres

DATABASE\_MIGRATION\_URL=... # 풀러 URL 을 쓰는 경우 직결 URL 을 별도로

# secrets

---

PAPERCLIP\_SECRETS\_MASTER\_KEY=... # base64 32바이트, local\_encrypted 마스터 키

# authenticated/public 모드 필수 (기본값은 local\_trusted + private)

```
PAPERCLIP_DEPLOYMENT_MODE=authenticated PAPERCLIP_DEPLOYMENT_EXPOSURE=public
BETTER_AUTH_SECRET=... # openssl rand -hex 32 PAPERCLIP_PUBLIC_URL=https://your-host #
auth.publicBaseUrl 과 동등, 명시적 base URL ``
```

authenticated/public 모드에서는 Better Auth 세션이 강제되며 BETTER\_AUTH\_SECRET 과 auth.publicBaseUrl 이 함께 필수다( cli/src/checks/deployment-auth-check.ts:28-60 ). env-only 호스팅을 만들려면 PAPERCLIP\_DEPLOYMENT\_MODE / EXPOSURE 도 명시해야 한다 — 미설정이면 local\_trusted + private 으로 떨어진다( server/src/config.ts:160-180 ).

local\_trusted 로 오래 사용하다 authenticated 로 전환하는 경우 — instance admin 이 local-board 한 명뿐일 때에 한해 — 부팅 시 1회용 board-claim URL 이 경고로 표시되어, 로그인한 사용자가 instance admin 으로 승격되고 local-board 권한이 강등된다( doc/DEPLOYMENT-MODES.md:115-126 , server/src/board-claim.ts:43-66 ).



# Pros & Cons — Paperclip의 트레이드오프

## 1. 평가 프레임

이 장은 한 가지 질문에 답한다 — *언제 Paperclip을 쓰고, 언제 쓰지 않아야 하는가?* 평가는 다음 6개 축에서 이뤄진다.

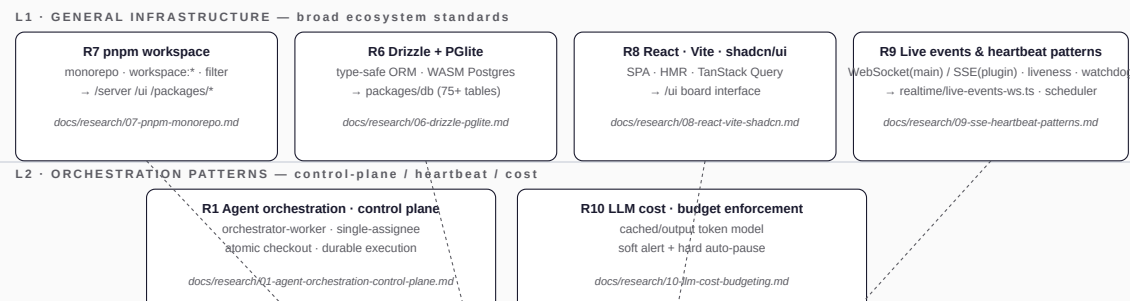
1. **개념적 명료성** — 무엇을 푸는지가 명확한가
2. **데이터 모델의 견고성** — 작업·비용·org가 일관되게 모델링됐는가
3. **운영 안전망** — 자동화 대비 인간 통제의 균형
4. **확장성** — 새 어댑터·플러그인을 코어 수정 없이 추가 가능한가
5. **운영 부담** — 설치·업데이트·모니터링의 무게
6. **성숙도** — 누락된 영역과 향후 리스크

# Research Map — 10편의 리서치로 본 Paperclip의 기술적 좌표

## 1. 4-층 위계

리서치 10편은 Paperclip 본체를 둘러싼 **4개의 동심층** 으로 정렬할 수 있다 — 가장 바깥의 **일반 인프라 표준** (R6\~R9) → **오케스트레이션 패턴** (R1, R10) → **Paperclip 코어** (이 리포트의 본체) → **실행 런타임** (R2\~R5). 그림 10이 그 위계를 한 페이지로 보여 준다.

그림 10. 리서치 ↔ Paperclip 4-층 위계 — 일반 인프라 표준 / 오케스트레이션 패턴 / Paperclip 코어 / 실행 런타임



# 1) 클론 · 실행

```
git clone https://github.com/paperclipai/paperclip.git && cd paperclip && pnpm install && pnpm dev
```

## 2) /companies 에서 회사 만들고

## 3) /agents/new 에서 CEO(claude\_local) 1명만 만들고

## 4) /goals 에서 사소한 골 1개 등록하고

## 5) /dashboard/live 에서 한 사이클을 실제로 실행

6) 동시에 /approvals ·  
/api/companies//costs/summary  
· activity\_log 확인



## 7) hard limit을 낮게 잡아 auto-pause 트리거 확인

...

이 한 사이클은 README와 `doc/SPEC.md`, 그리고 이 리포트가 문장만으로는 제공하지 못하는 *체험*의 마지막 한 조각을 제공한다.

## 6. 닫는 한 줄

Paperclip 은 단일 신기술이 아니라

5\~6개 표준 도구를 한 군데에 정합적으로 묶은 것

이다. 그 정합성이 곧 가치이며, 이 리포트의 10편 리서치가 그 정합의 외부 좌표를 표시한다.